

データマイニング特論

第8回

モデル評価・選択と不均衡データ

本科目シラバスの確認 3（授業計画）

#	日程	内容
第1回	4/16（木）	ガイダンス、イントロダクション+Python環境構築
第2回	4/23（木）	pandasによるデータ操作の基礎
第3回	4/30（木）	探索的データ分析（EDA）と可視化（オンデマンド）
第4回	5/7（木）	データの前処理と特徴量エンジニアリング
第5回	5/14（木）	分類の基礎 — k近傍法と決定木
第6回	5/21（木）	アンサンブル学習
第7回	5/28（木）	線形分類モデルと回帰
第8回	6/4（木）	モデル評価・選択と不均衡データ
第9回	6/11（木）	クラスタリング手法
第10回	6/18（木）	アソシエーション分析と異常検知
第11回	6/25（木）	テキストマイニング
第12回	7/2（木）	時系列データマイニング
第13回	7/9（木）	深層学習の概観と使いどころ
第14回	7/16（木）	公平性・倫理と再現性
第15回	7/23（木）	まとめ

【Point】 やむなく欠席した場合、資料を確認のうえ不明ところは聞いてください。

1

正解率のパラドックス

正解率だけでは何が見えないのか？



正解率では何が見えないのか？

【ベースラインは正解率 0.701 だがSHを1件も検出できない。】

正解率はベースライン以下 ⇒ でもSH再現率はずっと高い

手法	正解率	SH再現率
決定木	0.776	0.479
ランダムフォレスト (RF)	0.804	0.540
ロジスティック回帰	0.720	0.260
サポートベクターマシン (SVM, 線形)	0.764	0.915
ベースライン (全員を非SHと予測)	0.701	0.000

本日の環境準備 (I)

いろいろ試してみよう！

日本語対応のモジュールをインストールする(その1)

```
!pip install japanize-matplotlib
```

日本語対応のモジュールをインストールする(その2)

```
!apt-get install -y fonts-ipafont-gothic
```

各ライブラリーをインポートする

```
import numpy as np
```

```
import pandas as pd
```

```
import matplotlib.pyplot as plt
```

```
import japanize_matplotlib
```

```
import warnings
```

```
warnings.filterwarnings("ignore")
```

```
from sklearn.linear_model import LogisticRegression
```

```
from sklearn.ensemble import RandomForestClassifier
```

```
from sklearn.tree import DecisionTreeClassifier
```

```
from sklearn.svm import SVC
```

```
from sklearn.preprocessing import StandardScaler
```

```
from sklearn.model_selection import (train_test_split, StratifiedKFold, KFold, cross_validate)
```

```
from sklearn.metrics import (
```

```
    accuracy_score, classification_report,
```

```
    confusion_matrix, ConfusionMatrixDisplay,
```

```
    f1_score, recall_score, precision_score,
```

```
    roc_auc_score, average_precision_score,
```

```
    roc_curve, precision_recall_curve, auc
```

```
)
```

```
from imblearn.over_sampling import SMOTE
```

```
from imblearn.under_sampling import RandomUnderSampler
```

本日の環境準備（２）

いろいろ試してみよう！

#データ読み込み

```
URL = "https://data.insideairbnb.com/japan/kant%C5%8D/tokyo/2023-06-29/data/listings.csv.gz"
```

```
df = pd.read_csv(URL, compression="gzip", low_memory=False)
```

```
df["is_superhost"] = (df["host_is_superhost"] == "t").astype(int)
```

```
if df["price"].dtype == object:
```

```
    df["price"] = df["price"].str.replace(r"[$,]", "", regex=True).astype(float)
```

```
FEATURES = [
```

```
    "price", "number_of_reviews", "review_scores_rating",
```

```
    "accommodates", "bedrooms", "minimum_nights", "availability_365",
```

```
]
```

```
df_clean = df[FEATURES + ["is_superhost"]].dropna()
```

```
X = df_clean[FEATURES].values
```

```
y = df_clean["is_superhost"].values
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)
```

```
scaler = StandardScaler()
```

```
X_train_sc = scaler.fit_transform(X_train)
```

```
X_test_sc = scaler.transform(X_test)
```

```
X_sc = scaler.fit_transform(X) # 交差検証用
```

```
print(f"✓ データ準備完了:{len(df_clean):,} 件")
```

```
print(f"SH比率:{y.mean():.1%} 非SH:{1-y.mean():.1%}")
```

```
print(f"訓練:{len(X_train):,} テスト:{len(X_test):,}")
```

✓ データ準備完了：8,181 件

SH比率：29.8% 非SH：70.2%

訓練：6,544 テスト：1,637

正解率のパラドックス (1)

いろいろ試してみよう！

— ベースライン: 全員を非SH(多数クラス)と予測 —

```
y_pred_baseline = np.zeros(len(y_test), dtype=int) # 全員0(非SH)
```

```
baseline_acc = accuracy_score(y_test, y_pred_baseline)
```

```
baseline_rec = recall_score(y_test, y_pred_baseline, zero_division=0)
```

```
baseline_prec = precision_score(y_test, y_pred_baseline, zero_division=0)
```

```
baseline_f1 = f1_score(y_test, y_pred_baseline, zero_division=0)
```

```
print("— ベースライン(全員を非SHと予測)—")
```

```
print(f" 正解率 : {baseline_acc:.3f} ← 非SH比率そのもの")
```

```
print(f" SH再現率 : {baseline_rec:.3f} ← SHを1件も検出できていない")
```

```
print(f" SH F1 : {baseline_f1:.3f}")
```

— ロジスティック回帰(標準) —

```
lr = LogisticRegression(C=1.0, max_iter=1000, random_state=42)
```

```
lr.fit(X_train_sc, y_train)
```

```
y_pred_lr = lr.predict(X_test_sc)
```

```
lr_acc = accuracy_score(y_test, y_pred_lr)
```

```
lr_rec = recall_score(y_test, y_pred_lr, zero_division=0)
```

#次に続く。。

正解率のパラドックス (2)

いろいろ試してみよう！

```
# — 不均衡対処版(class_weight='balanced') —
# ※パラドックスは「対処を入れると正解率は下がるが再現率が激増する」形で最も鮮明になる
lr_bal = LogisticRegression(C=1.0, class_weight="balanced",
                             max_iter=1000, random_state=42)
lr_bal.fit(X_train_sc, y_train)
y_pred_bal = lr_bal.predict(X_test_sc)
bal_acc = accuracy_score(y_test, y_pred_bal)
bal_rec = recall_score(y_test, y_pred_bal, zero_division=0)

# 比較結果を「高い/低い」自動判定で表示(直書きしない)
def updown(value, ref):
    """value が ref より高い/低い/同等 を日本語で返す"""
    if value > ref + 1e-9:
        return f"ベースライン {ref:.3f} より高い"
    elif value < ref - 1e-9:
        return f"ベースライン {ref:.3f} より低い"
    else:
        return f"ベースライン {ref:.3f} とほぼ同じ"

print("\n— ロジスティック回帰(標準)—")
print(f"  正解率    : {lr_acc:.3f}  ← {updown(lr_acc, baseline_acc)}")
print(f"  SH再現率   : {lr_rec:.3f}  ← {updown(lr_rec, baseline_rec)}")
print("\n— ロジスティック回帰(class_weight='balanced')—")
print(f"  正解率    : {bal_acc:.3f}  ← {updown(bal_acc, baseline_acc)}")
print(f"  SH再現率   : {bal_rec:.3f}  ← {updown(bal_rec, baseline_rec)}")
print("  ※ 正解率はベースライン付近でも、SH再現率は大きく改善する点に注目")
```

#次に続く。。

正解率のパラドックス (3)

いろいろ試してみよう！

```
# — 比較グラフ(S3のビジュアルをコードで再現) —
# 3本(ベースライン / LR標準 / LR class_weight)で比較
fig, axes = plt.subplots(1, 2, figsize=(11, 4))
models_name = ["ベースライン¥n(全員非SH)", "ロジスティック¥n回帰(標準)", "ロジスティック¥n回帰(重み付け)"]
accs = [baseline_acc, lr_acc, bal_acc]
recs = [baseline_rec, lr_rec, bal_rec]
colors_acc = ["#F0A500", "#028090", "#5C4DB1"]
colors_rec = ["#E05252", "#028090", "#5C4DB1"]

for ax, vals, colors, title, ylim in [
    (axes[0], accs, colors_acc, "正解率の比較", (0.0, 1.0)),
    (axes[1], recs, colors_rec, "SH再現率の比較", (0.0, 1.0)),
]:
    bars = ax.bar(models_name, vals, color=colors, alpha=0.85, edgecolor="white", width=0.6)
    for bar, v in zip(bars, vals):
        ax.text(bar.get_x() + bar.get_width()/2,
                bar.get_height() + 0.01,
                f"{v:.3f}", ha="center", va="bottom",
                fontsize=11, fontweight="bold")
    ax.set_ylim(ylim)
    ax.set_title(title, fontsize=12)
    ax.grid(True, axis="y", alpha=0.3)
    ax.tick_params(axis="x", labelsize=9)

plt.suptitle("正解率のパラドックス: ¥n正解率がほぼ変わらなくてもSHを検出できるモデルの方が実用的", fontsize=12)
plt.tight_layout()
plt.savefig("accuracy_paradox.png", dpi=150, bbox_inches="tight")
plt.show()
```

正解率のパラドックス（４）

—— ベースライン（全員を非SHと予測） ——

正解率 : 0.701 ← 非SH比率そのもの
SH再現率 : 0.000 ← SHを1件も検出できていない
SH F1 : 0.000

—— ロジスティック回帰（標準） ——

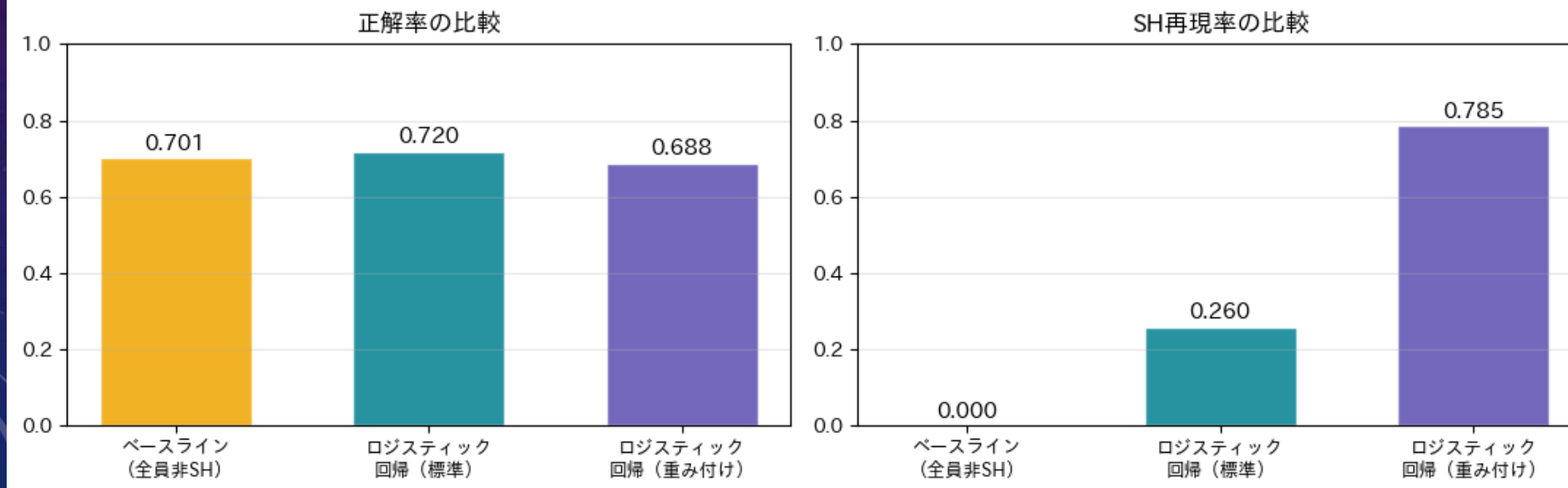
正解率 : 0.720 ← ベースライン 0.701 より高い
SH再現率 : 0.260 ← ベースライン 0.000 より高い

—— ロジスティック回帰（class_weight='balanced'） ——

正解率 : 0.688 ← ベースライン 0.701 より低い
SH再現率 : 0.785 ← ベースライン 0.000 より高い

※ 正解率はベースライン付近でも、SH再現率は大きく改善する点に注目

正解率のパラドックス：
正解率がほぼ変わらなくてもSHを検出できるモデルの方が実用的



正解率が「嘘をつく」仕組み

【Airbnb Tokyoデータのクラス比率】

非SH 70%
SH 30%

Step 1 全員を多数クラス（非SH）と予測する

何も考えず、すべての出力を「非SH」にする最も単純なモデルを想定する

Step 2 正解率を計算すると0.701

非SHが70%いるので、全員を非SHと予測するだけで70%正解できてしまう

Step 3 しかしSH再現率 = 0.000

本物のSHを1件も検出できていない。

正解率の高さは「多数クラスの正解率」に過ぎない。

正解率はクラス比率に引っ張られる指標。
SH再現率・F1・PR-AUCで補う必要がある。

混同行列の再整理

【混同行列 — 正解率に圧縮される前の情報】

混同行列は正解率に圧縮される前の情報。どのエラーが高コストかを決めてから指標を選ぶ。

予測：SH

予測：非SH

実際
SH

TP

真陽性
本物SHを
SHと正解

FP

偽陽性
非SHをSHと
誤判定

FN

偽陰性
SHを非SH
と見逃し

TN

真陰性
非SHを
非SHと正解

実際
非SH

正解率 多数クラスに引っ張られる
 $(TP + TN) / \text{全体}$

SH適合率 FPを減らしたい場合に重視
 $TP / (TP + FP)$

SH再現率 FNを減らしたい場合に重視
 $TP / (TP + FN)$

SH F1 適合率と再現率のバランス
 $2 \times P \times R / (P + R)$

混同行列の詳細確認（１）

いろいろ試してみよう！

#混同行列 - TP/FP/FN/TN

```
cm = confusion_matrix(y_test, y_pred_lr)
tn, fp, fn, tp = cm.ravel()
```

```
print("— ロジスティック回帰(標準)の混同行列 —")
```

```
print(f"  TP(本物SHをSHと正解)    : {tp:,}")
```

```
print(f"  FP(非SHをSHと誤判定)    : {fp:,}")
```

```
print(f"  FN(SHを非SHと見逃し)    : {fn:,} ← コストが高い!")
```

```
print(f"  TN(非SHを非SHと正解)    : {tn:,}")
```

```
print()
```

```
print(f"  正解率    = (TP+TN)/(全体) = ({tp}+{tn})/{tp+fp+fn+tn} = {(tp+tn)/(tp+fp+fn+tn):.3f}")
```

適合率・再現率は分母が0になり得るのでガード

```
prec_calc = tp/(tp+fp) if (tp+fp) > 0 else 0.0
```

```
rec_calc = tp/(tp+fn) if (tp+fn) > 0 else 0.0
```

```
print(f"  SH適合率 = TP/(TP+FP)    = {tp}/{tp+fp} = {prec_calc:.3f}")
```

```
print(f"  SH再現率 = TP/(TP+FN)    = {tp}/{tp+fn} = {rec_calc:.3f}")
```

```
#次に続く。。
```

混同行列の詳細確認（２）

いろいろ試してみよう！

混同行列の可視化(ベースライン / LR標準 / LR重み付けの3枚)

```
fig, axes = plt.subplots(1, 3, figsize=(15, 4.5))
```

```
cm_base = confusion_matrix(y_test, y_pred_baseline)
```

```
ConfusionMatrixDisplay(cm_base, display_labels=["非SH", "SH"]).plot(cmap="Oranges", ax=axes[0], colorbar=False)
```

```
axes[0].set_title(f"ベースライン¥nSH再現率 = {baseline_rec:.3f}", fontsize=11)
```

```
ConfusionMatrixDisplay(cm, display_labels=["非SH", "SH"]).plot(cmap="Blues", ax=axes[1], colorbar=False)
```

```
axes[1].set_title(f"ロジスティック回帰(標準)¥nSH再現率 = {lr_rec:.3f}", fontsize=11)
```

```
cm_bal = confusion_matrix(y_test, y_pred_bal)
```

```
ConfusionMatrixDisplay(cm_bal, display_labels=["非SH", "SH"]).plot(
```

```
    cmap="Purples", ax=axes[2], colorbar=False)
```

```
axes[2].set_title(f"ロジスティック回帰(重み付け)¥nSH再現率 = {bal_rec:.3f}", fontsize=11)
```

```
plt.suptitle("混同行列の比較:ベースライン vs 標準 vs 重み付け", fontsize=12)
```

```
plt.tight_layout()
```

```
plt.savefig("confusion_matrix_comparison.png", dpi=150, bbox_inches="tight")
```

```
plt.show()
```

混同行列の詳細確認（3）

—— ロジスティック回帰（標準）の混同行列 ——

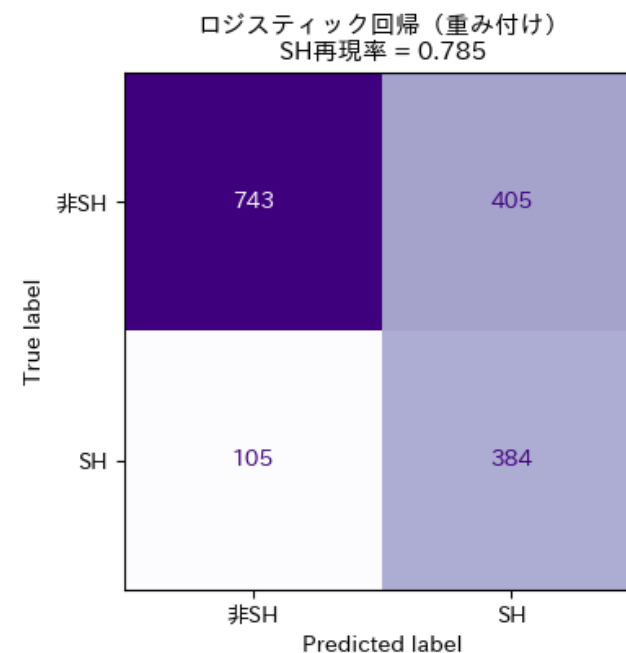
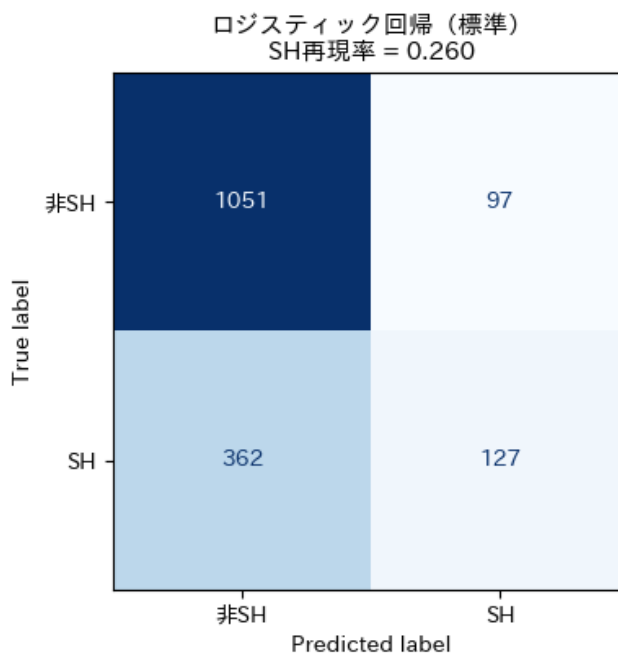
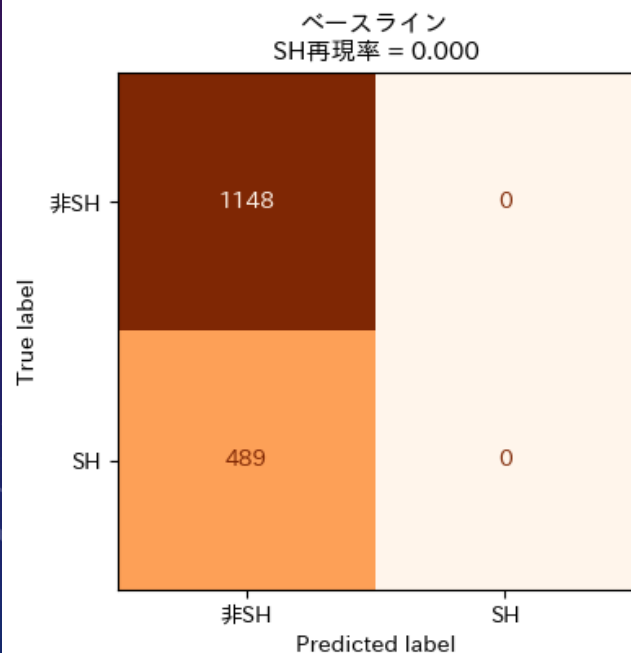
TP（本物SHをSHと正解）：127
 FP（非SHをSHと誤判定）：97
 FN（SHを非SHと見逃し）：362 ← コストが高い！
 TN（非SHを非SHと正解）：1,051

正解率 = $(TP+TN)/(全体) = (127+1051)/1637 = 0.720$

SH適合率 = $TP/(TP+FP) = 127/(127+97) = 0.567$

SH再現率 = $TP/(TP+FN) = 127/(127+362) = 0.260$

混同行列の比較：ベースライン vs 標準 vs 重み付け



評価指標の選び方（I）

【どの指標をいつ使うか — 目的で選ぶ】

適合率 (Precision, P) $TP / (TP + FP)$

「SHと予測したものが本当にSH」かを確認したいとき
使う場面：FPのコストが高い場面

再現率 (Recall, R) $TP / (TP + FN)$

「本物のSHを取りこぼさない」ことを優先したいとき
使う場面：FNのコストが高い場面

F1 $2 \times P \times R / (P + R)$

適合率と再現率のバランスを取りたいとき
使う場面：どちらか一方に偏りたくない

ROC-AUC 全閾値での判別能力

均衡データでモデルの総合力を評価するとき
使う場面：均衡データ向き

PR-AUC $P \times R$ 曲線の下面積

不均衡データで少数クラスの検出性能を評価するとき
使う場面：不均衡データ必須

不均衡データのAirbnbスーパーホスト判定にはPR-AUCが最も信頼できる指標

評価指標の選び方（2）

いろいろ試してみよう！

#5つの評価指標を一括確認

```
probs_lr = lr.predict_proba(X_test_sc)[: , 1]

print("— classification_report(P・R・F1)—")
print(classification_report(y_test, y_pred_lr,
    target_names=["非SH", "SH"]))

roc_lr = roc_auc_score(y_test, probs_lr)
pr_lr = average_precision_score(y_test, probs_lr)

print(f"ROC-AUC   : {roc_lr:.3f}   (ベースライン = 0.500 固定)")
print(f"PR-AUC    : {pr_lr:.3f}   (ベースライン = SH比率 ≈ {y_test.mean():.3f})")
print()
print("— 各指標の使い分け —")
print("  SH適合率(P): SHと予測したものが本当にSHかを確認したいとき")
print("  SH再現率(R): 本物SHを取りこぼさないことを優先したいとき")
print("  SH F1       : PとRのバランスをとりたいとき")
print("  ROC-AUC     : 均衡データで総合力を評価したいとき")
print("  PR-AUC ★    : 不均衡データで少数クラスの検出性能を評価したいとき")
```

評価指標の選び方（3）

—— classification_report (P・R・F1) ——

	precision	recall	f1-score	support
非SH	0.74	0.92	0.82	1148
SH	0.57	0.26	0.36	489
accuracy			0.72	1637
macro avg	0.66	0.59	0.59	1637
weighted avg	0.69	0.72	0.68	1637

ROC-AUC : 0.760 (ベースライン = 0.500 固定)

PR-AUC : 0.545 (ベースライン = SH比率 $\approx$ 0.299)

—— 各指標の使い分け ——

SH適合率 (P) : SHと予測したものが本当にSHかを確認したいとき

SH再現率 (R) : 本物SHを取りこぼさないことを優先したいとき

SH F1 : PとRのバランスをとりたいとき

ROC-AUC : 均衡データで総合力を評価したいとき

PR-AUC ★ : 不均衡データで少数クラスの検出性能を評価したいとき

閾値のトレードオフ（1）

【閾値を動かすと何が起こるか】

デフォルトの閾値0.5：predict_proba() の確率が 0.5 以上なら SH と判定

⇒閾値はpredict_proba()の確率に任意の値を設定できる。目的に合わせた調整が可能

閾値	0.3（低い）	0.5（標準）	0.7（高い）
SH適合率	低下	中間	上昇
SH再現率	上昇	中間	低下
SH F1	要確認	バランス	要確認
特徴	より多くをSHと判定 見逃しは減るが誤検出が増える	scikit-learnのデフォルト まず最初に確認する値	SHと判定するのを絞り込む 誤検出は減るが見逃しが増える

閾値のトレードオフ (2)

#閾値のトレードオフ

```
print("— 3つの閾値で指標を比較 —")
thresholds_compare = [0.3, 0.5, 0.7]
rows_thr = []
for thr in thresholds_compare:
    pred_thr = (probs_lr >= thr).astype(int)
    rows_thr.append({
        "閾値": thr,
        "SH適合率(P)": round(precision_score(y_test, pred_thr, zero_division=0), 3),
        "SH再現率(R)": round(recall_score(y_test, pred_thr, zero_division=0), 3),
        "SH F1": round(f1_score(y_test, pred_thr, zero_division=0), 3),
        "正解率": round(accuracy_score(y_test, pred_thr), 3),
    })
print(pd.DataFrame(rows_thr).to_string(index=False))
```

最適閾値の計算(F1最大)

```
prec_arr, rec_arr, thr_arr = precision_recall_curve(y_test, probs_lr)
f1_arr = 2 * prec_arr[:-1] * rec_arr[:-1] / (prec_arr[:-1] + rec_arr[:-1] + 1e-8)
best_thr = thr_arr[f1_arr.argmax()]
print(f"¥nF1スコアが最大になる最適閾値:{best_thr:.3f}")
```

#次に続く。。。

閾値のトレードオフ (3)

閾値トレードオフのグラフ

```
fig, axes = plt.subplots(1, 2, figsize=(12, 4.5))
axes[0].plot(thr_arr, prec_arr[:-1], color="#028090", linewidth=2, label="SH適合率(P)")
axes[0].plot(thr_arr, rec_arr[:-1], color="#E05252", linewidth=2, label="SH再現率(R)")
for thr, ls, lbl in [(0.3, "--", "0.3"), (0.5, "-", "0.5(標準)"), (0.7, ":", "0.7")]:
    axes[0].axvline(x=thr, color="gray", linestyle=ls, linewidth=1.2, alpha=0.7, label=f"閾値={lbl}")
axes[0].set_xlabel("閾値", fontsize=11)
axes[0].set_ylabel("スコア", fontsize=11)
axes[0].set_title("閾値 vs SH適合率(P)・再現率(R)", fontsize=11)
axes[0].legend(fontsize=9)
axes[0].grid(True, alpha=0.3)

axes[1].plot(thr_arr, f1_arr, color="#5C4DB1", linewidth=2, label="SH F1")
axes[1].axvline(x=0.5, color="gray", linestyle="-", linewidth=1.5, label="標準閾値 0.5")
axes[1].axvline(x=best_thr, color="#F0A500", linestyle="--", linewidth=1.5, label=f"最適閾値 {best_thr:.2f}")
axes[1].set_xlabel("閾値", fontsize=11)
axes[1].set_ylabel("SH F1スコア", fontsize=11)
axes[1].set_title("閾値 vs SH F1スコア", fontsize=11)
axes[1].legend(fontsize=10)
axes[1].grid(True, alpha=0.3)

plt.suptitle("閾値のトレードオフ:PとRは逆に動く", fontsize=12)
plt.tight_layout()
plt.savefig("threshold_tradeoff.png", dpi=150, bbox_inches="tight")
plt.show()
```

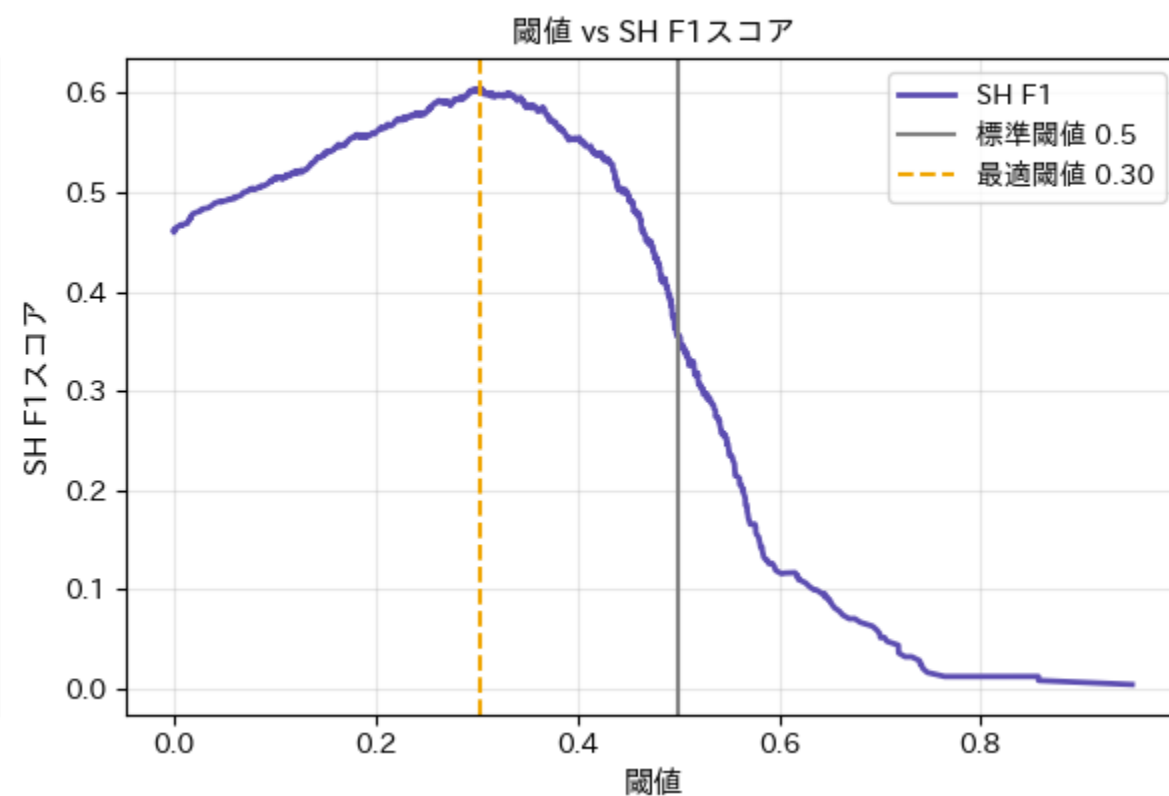
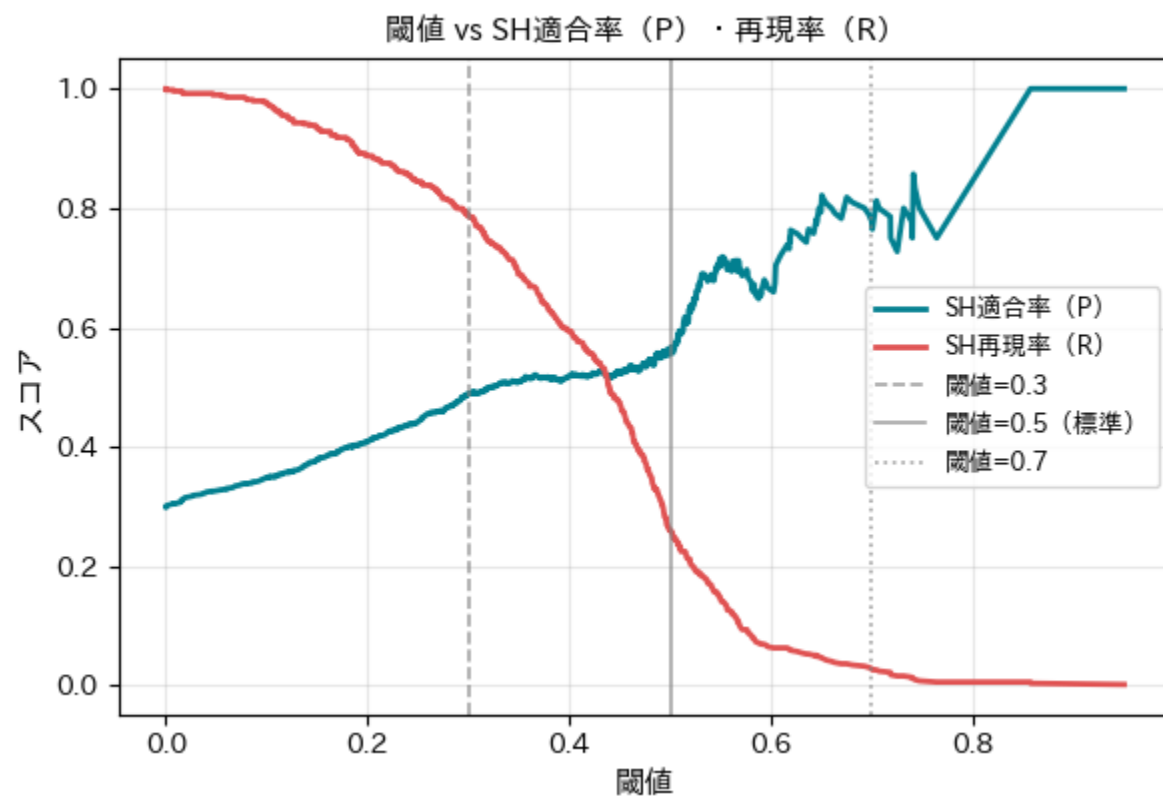
閾値のトレードオフ (4)

— 3つの閾値で指標を比較 —

閾値	SH適合率 (P)	SH再現率 (R)	SH F1	正解率
0.3	0.489	0.785	0.602	0.690
0.5	0.567	0.260	0.356	0.720
0.7	0.765	0.027	0.051	0.707

F1スコアが最大になる最適閾値：0.303

閾値のトレードオフ：PとRは逆に動く



ROC曲線とPR曲線（Ⅰ）

#第5～7回の主要手法を準備

```
dt = DecisionTreeClassifier(max_depth=4, random_state=42)
rf = RandomForestClassifier(n_estimators=300, random_state=42, n_jobs=-1)
```

不均衡データで多数クラスに倒れるのを防ぐため `class_weight='balanced'` を付与

```
svm = SVC(kernel="linear", C=1.0, class_weight="balanced", probability=True, random_state=42)
```

```
dt.fit(X_train, y_train)
```

```
rf.fit(X_train, y_train)
```

```
svm.fit(X_train_sc, y_train)
```

```
models_eval = [
    ("決定木", dt, X_test, "--", "#636E72"),
    ("ランダムフォレスト", rf, X_test, "-", "#27AE60"),
    ("ロジスティック回帰", lr, X_test_sc, "-", "#028090"),
    ("SVM(線形・重み付け)", svm, X_test_sc, "-", "#5C4DB1"),
]
```

```
fig, axes = plt.subplots(1, 2, figsize=(13, 5))
```

```
baseline_pr = y_test.mean()
```

```
print("— ROC-AUC と PR-AUC の一覧 —")
```

```
for name, model, X_inp, ls, color in models_eval:
```

```
    probs = model.predict_proba(X_inp)[: , 1]
```

```
    roc_a = roc_auc_score(y_test, probs)
```

```
    pr_a = average_precision_score(y_test, probs)
```

```
    print(f" {name:18s} ROC-AUC={roc_a:.3f} PR-AUC={pr_a:.3f}")
```

#次に続く。。。

ROC曲線とPR曲線（2）

```
fpr, tpr, _ = roc_curve(y_test, probs)
axes[0].plot(fpr, tpr, color=color, linestyle=ls,
             linewidth=1.8, label=f"{name} ({roc_a:.3f})")

p_arr, r_arr, _ = precision_recall_curve(y_test, probs)
axes[1].plot(r_arr, p_arr, color=color, linestyle=ls,
             linewidth=1.8, label=f"{name} ({pr_a:.3f})")

axes[0].plot([0,1],[0,1],"k--",alpha=0.4,label="ランダム(0.500)")
axes[0].set_xlabel("偽陽性率(FPR)", fontsize=11)
axes[0].set_ylabel("真陽性率(TPR = 再現率)", fontsize=11)
axes[0].set_title("ROC曲線(均衡データ向き)", fontsize=11)
axes[0].legend(fontsize=9, loc="lower right")
axes[0].grid(True, alpha=0.3)

axes[1].axhline(y=baseline_pr, color="k", linestyle="--",
               alpha=0.6, label=f"ベースライン({baseline_pr:.3f})")
axes[1].set_xlabel("再現率(R = Recall)", fontsize=11)
axes[1].set_ylabel("適合率(P = Precision)", fontsize=11)
axes[1].set_title("PR曲線(不均衡データ向き ★)", fontsize=11)
axes[1].legend(fontsize=9, loc="upper right")
axes[1].grid(True, alpha=0.3)

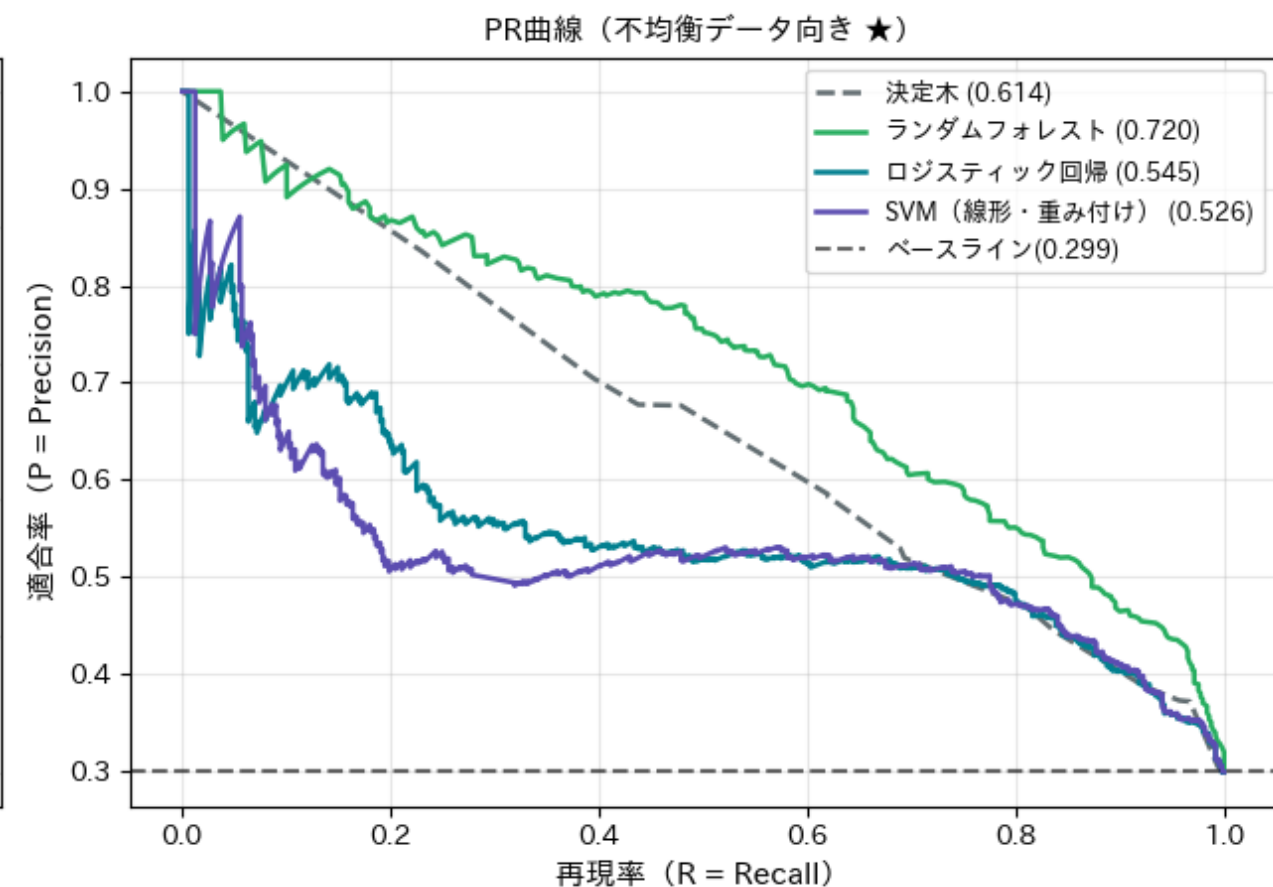
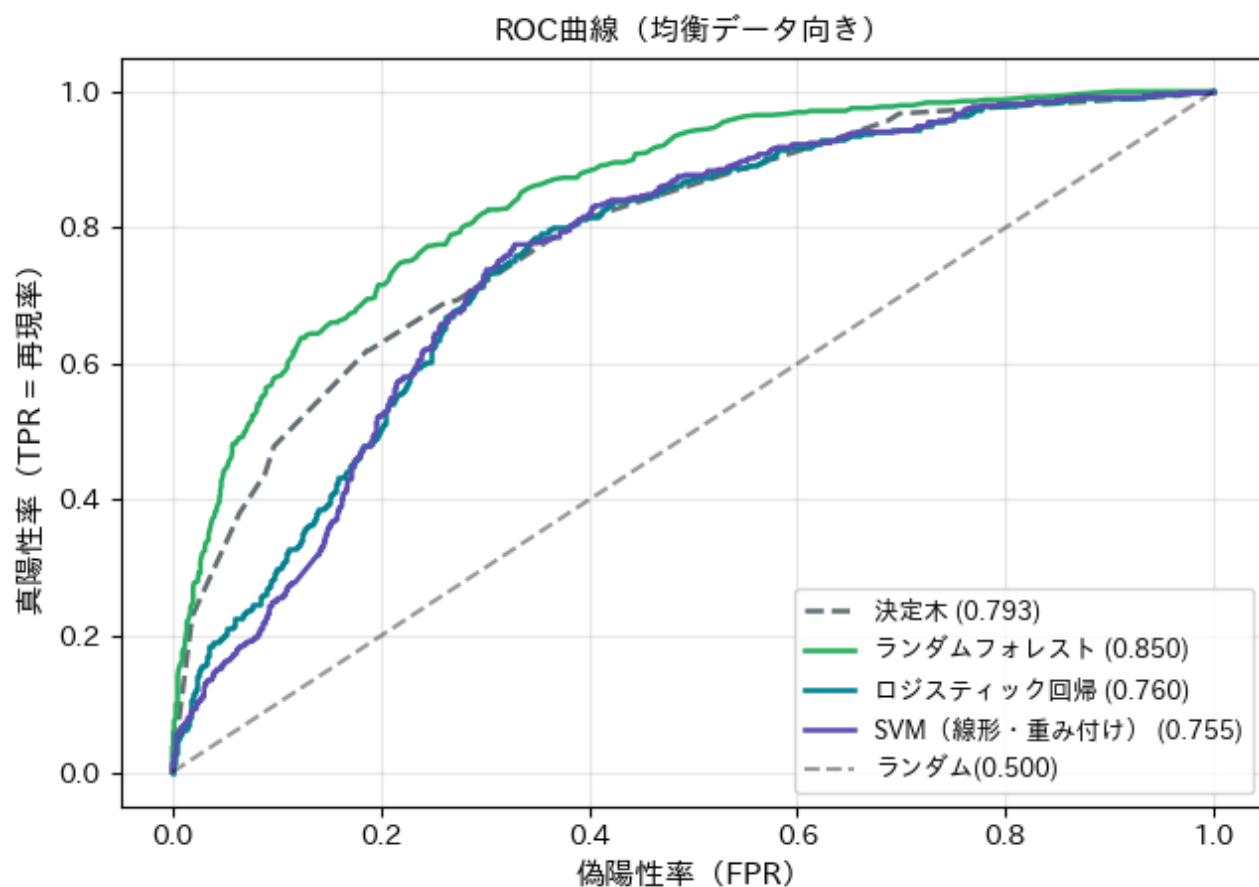
plt.suptitle("ROC曲線 vs PR曲線:全手法の比較", fontsize=12)
plt.tight_layout()
plt.savefig("roc_pr_curves.png", dpi=150, bbox_inches="tight")
plt.show()
```

ROC曲線とPR曲線（3）

— ROC-AUC と PR-AUC の一覧 —

決定木	ROC-AUC=0.793	PR-AUC=0.614
ランダムフォレスト	ROC-AUC=0.850	PR-AUC=0.720
ロジスティック回帰	ROC-AUC=0.760	PR-AUC=0.545
SVM（線形・重み付け）	ROC-AUC=0.755	PR-AUC=0.526

ROC曲線 vs PR曲線：全手法の比較



2

不均衡データへの 3つの対処法

PR-AUCで効果を比較できる



不均衡データへの3つの対処法

【問題：SH 30% vs 非SH 70%】

→ 損失関数が多数クラスに引っ張られ、少数クラスの学習が不十分になる】

①クラス重み付け (`class_weight = 'balanced'`)

少数クラスのサンプルに大きな重みをかけて損失関数を補正

○ 引数1つ追加するだけ × データ量は変わらない

⇒ほぼすべての手法に使える

②オーバーサンプリング (SMOTE)

少数クラスの近傍間を補間して人工サンプルを生成しクラス比率を揃える

○ データ量が保たれる × 合成データへの過学習リスク

⇒情報は失われない

③アンダーサンプリング (`RandomUnderSampler`)

多数クラスのサンプルをランダムに削除してクラス比率を揃える

○ 計算が速い × 多数クラスの情報失われる

⇒大規模データに向く

実務の手順：まず①を試す → 効果が足りなければ②へ → 大規模データなら③も選択肢

不均衡データへの対処（クラス重み付け）（Ⅰ）

いろいろ試してみよう！

#クラス重み付け

```
def metrics(name, model, X_inp, y_true):
    pred = model.predict(X_inp)
    proba = model.predict_proba(X_inp)[: , 1]
    return {
        "手法": name,
        "正解率": round(accuracy_score(y_true, pred), 3),
        "SH適合率P": round(precision_score(y_true, pred, zero_division=0), 3),
        "SH再現率R": round(recall_score(y_true, pred, zero_division=0), 3),
        "SH F1": round(f1_score(y_true, pred, zero_division=0), 3),
        "PR-AUC": round(average_precision_score(y_true, proba), 3),
        "ROC-AUC": round(roc_auc_score(y_true, proba), 3),
    }

cw_df = pd.DataFrame([
    metrics("LR(対処なし)", lr, X_test_sc, y_test),
    metrics("LR(class_weight)", lr_bal, X_test_sc, y_test),
]).set_index("手法")
print(cw_df.to_string())
```

不均衡データへの対処（クラス重み付け）（2）

手法	正解率	SH適合率P	SH再現率R	SH F1	PR-AUC	ROC-AUC
LR（対処なし）	0.720	0.567	0.260	0.356	0.545	0.76
LR（class_weight）	0.688	0.487	0.785	0.601	0.542	0.76

不均衡データへの対処（リサンプリング）（Ⅰ）

いろいろ試してみよう！

リサンプリング

```
sm = SMOTE(random_state=42)
X_sm, y_sm = sm.fit_resample(X_train_sc, y_train)
print(f" Before : SH={y_train.sum():,} / 全{len(y_train):,}")
print(f" After  : SH={y_sm.sum():,} / 全{len(y_sm):,}")
lr_smote = LogisticRegression(C=1.0, max_iter=1000, random_state=42)
lr_smote.fit(X_sm, y_sm)

print("\n— ③ アンダーサンプリング(多数クラスを減らす)—")
rus = RandomUnderSampler(random_state=42)
X_us, y_us = rus.fit_resample(X_train_sc, y_train)
print(f" Before : 非SH={len(y_train)-y_train.sum():,} / 全{len(y_train):,}")
print(f" After  : 非SH={len(y_us)-y_us.sum():,} / 全{len(y_us):,}")
lr_under = LogisticRegression(C=1.0, max_iter=1000, random_state=42)
lr_under.fit(X_us, y_us)

print("\n— 4条件の比較 —")
resamp_df = pd.DataFrame([
    metrics("①標準(対処なし)", lr, X_test_sc, y_test),
    metrics("②class_weight=balanced", lr_bal, X_test_sc, y_test),
    metrics("③SMOTE", lr_smote, X_test_sc, y_test),
    metrics("④アンダーサンプリング", lr_under, X_test_sc, y_test),
]).set_index("手法")
print(resamp_df.to_string())
```

#次に続く。。

不均衡データへの対処（リサンプリング）（2）

いろいろ試してみよう！

PR-AUCで比較グラフ

```
fig, ax = plt.subplots(figsize=(9, 4.5))
colors_bar = ["#636E72", "#028090", "#5C4DB1", "#E05252"]
bars = ax.bar(resamp_df.index, resamp_df["PR-AUC"],
              color=colors_bar, alpha=0.85, edgecolor="white", width=0.55)
ax.axhline(y=baseline_pr, color="black", linestyle="--",
          linewidth=1.5, label=f"ベースライン({baseline_pr:.3f})")
for bar in bars:
    ax.text(bar.get_x()+bar.get_width()/2, bar.get_height()+0.005,
          f"{bar.get_height():.3f}", ha="center", va="bottom",
          fontsize=11, fontweight="bold")
ax.set_ylim(0, max(resamp_df["PR-AUC"])*1.18)
ax.set_ylabel("PR-AUC", fontsize=11)
ax.set_title("不均衡対処法の比較(PR-AUCで評価)", fontsize=12)
ax.legend(fontsize=10)
ax.grid(True, axis="y", alpha=0.3)
plt.tight_layout()
plt.savefig("resampling_pr_auc.png", dpi=150, bbox_inches="tight")
plt.show()
```

不均衡データへの対処（リサンプリング）（3）

—— ② SMOTE（少数クラスを増やす） ——

Before : SH=1,953 / 全6,544

After : SH=4,591 / 全9,182

—— ③ アンダーサンプリング（多数クラスを減らす） ——

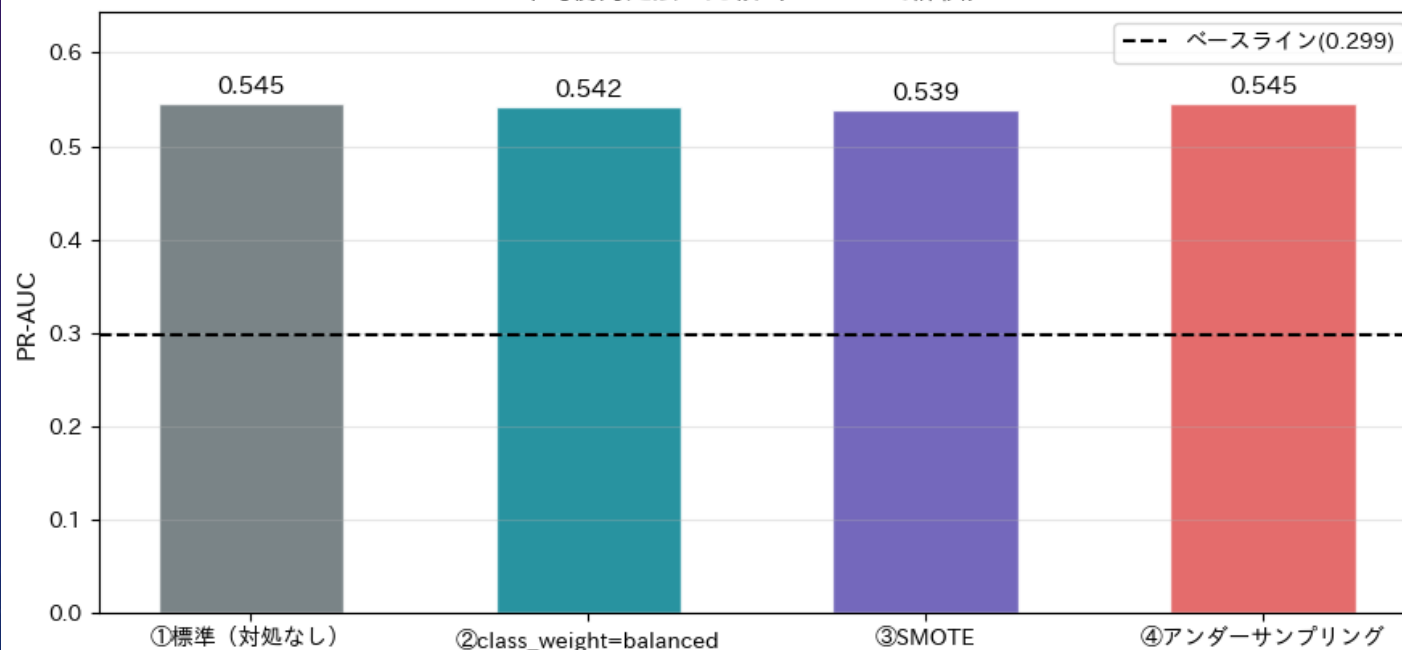
Before : 非SH=4,591 / 全6,544

After : 非SH=1,953 / 全3,906

—— 4条件の比較 ——

手法	正解率	SH適合率P	SH再現率R	SH F1	PR-AUC	ROC-AUC
①標準（対処なし）	0.720	0.567	0.260	0.356	0.545	0.760
②class_weight=balanced	0.688	0.487	0.785	0.601	0.542	0.760
③SMOTE	0.684	0.482	0.783	0.597	0.539	0.759
④アンダーサンプリング	0.692	0.490	0.789	0.605	0.545	0.760

不均衡対処法の比較（PR-AUCで評価）



Stratified K-Fold 交差検証 (I)

```
# K-Fold vs Stratified K-Fold
```

```
kf = KFold(n_splits=5, shuffle=True, random_state=42)
```

```
skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
```

```
print("— 各Foldのクラス比率(SH比率)—")
```

```
for cv_name, cv_obj in [("通常 K-Fold", kf),  
                        ("Stratified K-Fold", skf)]:
```

```
    ratios = [y[val].mean()
```

```
               for _, val in cv_obj.split(X_sc, y)]
```

```
    print(f" {cv_name}: {[round(r,3) for r in ratios]}")
```

```
    print(f"          平均={np.mean(ratios):.3f} "
```

```
          f"標準偏差={np.std(ratios):.4f}")
```

```
print()
```

```
print("— Stratified K-Fold で精度推定 —")
```

```
cv_models = {
```

```
    "LR(対処なし)": LogisticRegression(C=1.0, max_iter=1000),
```

```
    "LR(class_weight)": LogisticRegression(C=1.0, class_weight="balanced",  
                                           max_iter=1000),
```

```
    "RF(class_weight)": RandomForestClassifier(n_estimators=200,  
                                             class_weight="balanced",  
                                             random_state=42, n_jobs=-1),
```

```
}
```

```
#次に続く。。。
```

Stratified K-Fold 交差検証 (2)

```
cv_results = []
for name, model in cv_models.items():
    sc = cross_validate(
        model, X_sc, y, cv=skf,
        scoring=["f1", "average_precision", "roc_auc"],
        n_jobs=-1
    )
    cv_results.append({
        "手法": name,
        "F1": f"{sc['test_f1'].mean():.3f}±{sc['test_f1'].std():.3f}",
        "PR-AUC": f"{sc['test_average_precision'].mean():.3f}"
            f"±{sc['test_average_precision'].std():.3f}",
        "ROC-AUC": f"{sc['test_roc_auc'].mean():.3f}"
            f"±{sc['test_roc_auc'].std():.3f}",
    })
cv_df = pd.DataFrame(cv_results).set_index("手法")
print(cv_df.to_string())
print("※(±は5FoldのStd: 小さいほど評価が安定)")
```

Stratified K-Fold 交差検証 (3)

—— 各Foldのクラス比率 (SH比率) ——

通常 K-Fold : [np.float64(0.297), np.float64(0.315), np.float64(0.304), np.float64(0.285), np.float64(0.29)]
平均=0.298 標準偏差=0.0105

Stratified K-Fold: [np.float64(0.299), np.float64(0.299), np.float64(0.298), np.float64(0.298), np.float64(0.298)]
平均=0.298 標準偏差=0.0003

—— Stratified K-Fold で精度推定 ——

	F1	PR-AUC	ROC-AUC
手法			
LR (対処なし)	0.353±0.015	0.540±0.018	0.756±0.009
LR (class_weight)	0.591±0.008	0.538±0.018	0.755±0.008
RF (class_weight)	0.630±0.016	0.727±0.018	0.852±0.011

(±は5FoldのStd: 小さいほど評価が安定)

第5～8回の全手法比較（Ⅰ）

いろいろ試してみよう！

最終比較表

```
rf_bal = RandomForestClassifier(n_estimators=300, class_weight="balanced", random_state=42, n_jobs=-1)
rf_bal.fit(X_train, y_train)
```

```
try:
```

```
    import xgboost as xgb
    xgb_m = xgb.XGBClassifier(n_estimators=200, learning_rate=0.1,
                             max_depth=4, subsample=0.8,
                             eval_metric="logloss", random_state=42,
                             n_jobs=-1, verbosity=0)
```

```
    xgb_m.fit(X_train, y_train)
```

```
    has_xgb = True
```

```
except ImportError:
```

```
    has_xgb = False
```

```
all_rows = [
```

```
    metrics("決定木", dt, X_test, y_test),
```

```
    metrics("ランダムフォレスト", rf, X_test, y_test),
```

```
    metrics("ロジスティック回帰", lr, X_test_sc, y_test),
```

```
    metrics("LR + class_weight", lr_bal, X_test_sc, y_test),
```

```
    metrics("RF + class_weight", rf_bal, X_test, y_test),
```

```
    metrics("LR + SMOTE", lr_smote, X_test_sc, y_test),
```

```
    metrics("アンダーサンプリング", lr_under, X_test_sc, y_test),
```

```
    metrics("SVM(線形・重み付け)", svm, X_test_sc, y_test),
```

```
]
```

#次に続く。。。

第5～8回の全手法比較（2）

いろいろ試してみよう！

```
if has_xgb:
    all_rows.insert(2, metrics("XGBoost", xgb_m, X_test, y_test))

final_df = pd.DataFrame(all_rows).set_index("手法")
print(final_df.to_string())

print(f"¥nベースライン正解率   : {baseline_acc:.3f}")
print(f"PR-AUCのベースライン: {baseline_pr:.3f}")
print(f"¥nPR-AUCランキング(高い順):")
print(final_df["PR-AUC"].sort_values(ascending=False).to_string())
```

最終比較グラフ

```
fig, axes = plt.subplots(1, 2, figsize=(14, 5.5))
x = np.arange(len(final_df))
w = 0.38

axes[0].bar(x-w/2, final_df["正解率"], w, label="正解率", color="#4A6274", alpha=0.85)
axes[0].bar(x+w/2, final_df["SH再現率R"], w, label="SH再現率R", color="#E05252", alpha=0.85)
axes[0].axhline(y=baseline_acc, color="#F0A500", linestyle="--",
                linewidth=1.5, label=f"ベースライン正解率({baseline_acc:.3f})")
axes[0].set_xticks(x)
axes[0].set_xticklabels(final_df.index, rotation=20, ha="right", fontsize=9)
axes[0].set_ylim(0.0, 1.0)
axes[0].legend(fontsize=9)
axes[0].grid(True, axis="y", alpha=0.3)
axes[0].set_title("正解率 vs SH再現率R", fontsize=11)
```

#次に続く。。

第5～8回の全手法比較（3）

いろいろ試してみよう！

```
axes[1].bar(x-w/2, final_df["ROC-AUC"],w, label="ROC-AUC", color="#F0A500", alpha=0.85)
axes[1].bar(x+w/2, final_df["PR-AUC"], w, label="PR-AUC", color="#E05252", alpha=0.85)
axes[1].axhline(y=baseline_pr, color="black", linestyle="--",
                linewidth=1.5, label=f"PR-AUCベースライン({baseline_pr:.3f})")
axes[1].set_xticks(x)
axes[1].set_xticklabels(final_df.index, rotation=20, ha="right", fontsize=9)
axes[1].set_ylim(0.0, 1.0)
axes[1].legend(fontsize=9)
axes[1].grid(True, axis="y", alpha=0.3)
axes[1].set_title("ROC-AUC vs PR-AUC", fontsize=11)

plt.suptitle("第5～8回 全手法の最終比較(不均衡対処の効果を確認)",
            fontsize=12, fontweight="bold")
plt.tight_layout()
plt.savefig("final_comparison.png", dpi=150, bbox_inches="tight")
plt.show()
```

第5～8回の全手法比較（4）

手法	正解率	SH適合率P	SH再現率R	SH F1	PR-AUC	ROC-AUC
決定木	0.776	0.676	0.479	0.560	0.614	0.793
ランダムフォレスト		0.804	0.733	0.540	0.622	0.720
XGBoost	0.787	0.692	0.515	0.591	0.708	0.844
ロジスティック回帰		0.720	0.567	0.260	0.356	0.545
LR + class_weight	0.688	0.487	0.785	0.601	0.542	0.760
RF + class_weight	0.808	0.748	0.540	0.627	0.727	0.850
LR + SMOTE	0.684	0.482	0.783	0.597	0.539	0.759
アンダーサンプリング		0.692	0.490	0.789	0.605	0.545
SVM（線形・重み付け）		0.665	0.466	0.822	0.595	0.526

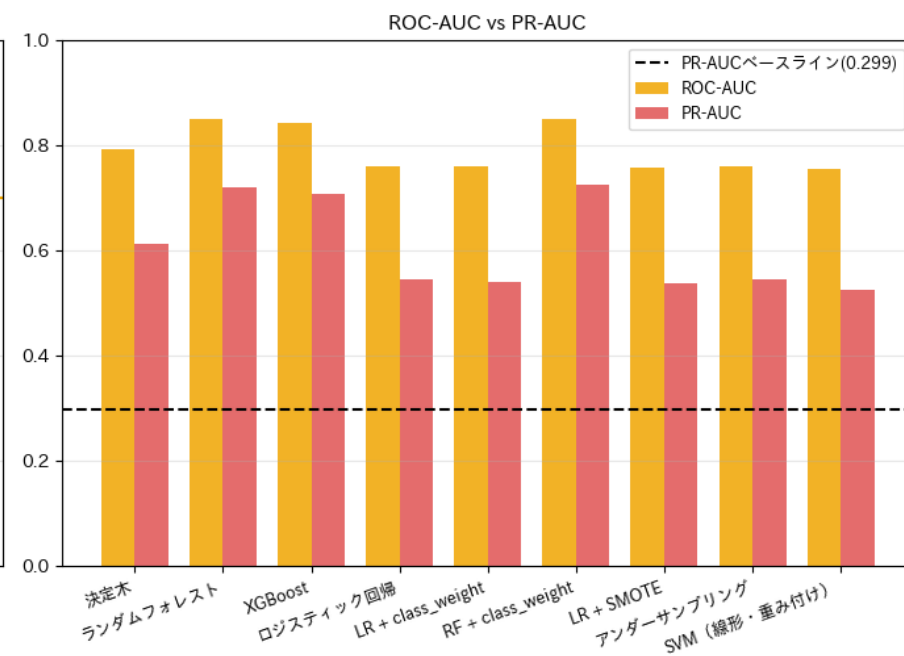
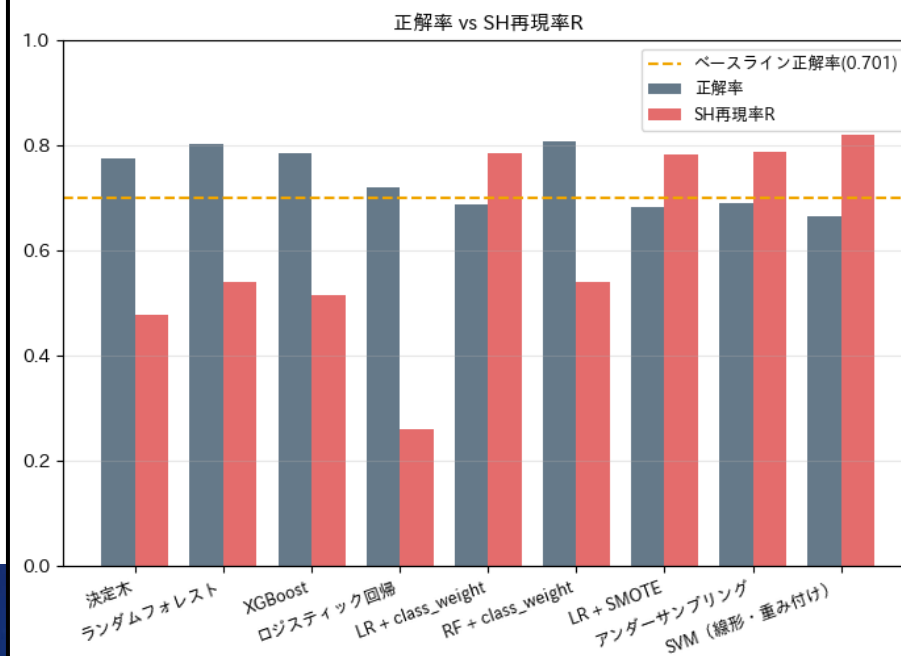
ベースライン正解率 : 0.701

PR-AUCのベースライン: 0.299

PR-AUCランキング（高い順）:

手法	PR-AUC
RF + class_weight	0.727
ランダムフォレスト	0.720
XGBoost	0.708
決定木	0.614
ロジスティック回帰	0.545
アンダーサンプリング	0.545
LR + class_weight	0.542
LR + SMOTE	0.539
SVM（線形・重み付け）	0.526

第5～8回 全手法の最終比較（不均衡対処の効果を確認）



3

本日の課題

頑張ってください。



課題

【課題】 期限：2026/6/10（水） 23：59まで

WebClassの『データマイニング特論』アクセスし、
本日の日付が記載される「課題」を期限内に実施してください。

・ WebClass -> データマイニング特論 -> 【第8回（6/4）】課題

（課題1） 本日の演習で「いろいろ試してみよう！」の中で実施した入出力コードを
html形式でエクスポートし、提出しなさい。

html形式にする手順

- ①ipynb形式でダウンロード（ファイル名の例：aaa.ipynb）
- ②再び「ファイル」へアップロード
- ③以下のコマンドを入力

```
!jupyter nbconvert --to html "aaa.ipynb"
```

【ADSセミナー】

本学の「先端データ科学研究センター」が主催する
セミナーが6/25（木）から4週にわたって
開催されます。

内容が盛りだくさんなので、**研究室を選択する前の
参考になる**かと思います。

事前登録フォームから申し込みあり。

【Point】 普段の授業では聞けない話もあるかも！？

ADSセミナー

対象 東京情報大学
全学部・学年/大学院生/教職員

会場 1号館 201教室

【第1回】 6/25（木） 16:30 - 18:00

講師： 釣井 達也 先生
総合情報学科 准教授（数理情報研究ユニット）



量子ウォークに潜む群構造について

量子ウォークはランダムウォークの量子版と解釈される数学モデルであり、その振る舞いはランダムウォークと大きく異なります。本セミナーでは、量子ウォークの簡単な具体例を通してランダムウォークとの違いを観察します。特に、量子ウォークに潜む群構造に着目し、この群構造を活用して、量子ウォークの特徴的な挙動のひとつである「周期」を求める方法について紹介します。

【第2回】 7/2（木） 16:30 - 18:00

講師： 早稲田 篤志 先生
総合情報学科 准教授（情報セキュリティ研究ユニット）



個人情報保護と機械学習

プライバシーに関する情報は解析して使用することで有益な情報を得られることが期待できます。一方、漏洩等で外部に漏れると大きな問題となることがあります。そこで、基本的なプライバシー保護の指標を紹介し、機械学習との関係を考察します。

【第3回】 7/9（木） 16:30 - 18:00

講師： 西村 明 先生
総合情報学科 教授（情報メディア学系）



音響法科学（オーディオ・フォレンジクス）

音響法科学は、音響学の原理と技術を、犯罪捜査・法廷および不正等の検知のために活かす研究と学問を指します。話者識別、雑音除去と音声強調、録音属性推定に大別され、本講演では、前二者を概観したのち、AI音声の判別を含む後者の研究について概説します。

【第4回】 7/16（木） 16:30 - 18:00

講師： 小早川 睦貴 先生
総合情報学科 准教授（生命情報研究ユニット）



身体情報から読みとる心の働き

心理学的研究で心を読み取る際には、さまざまな人間の「反応」を測ることが不可欠です。一般的には反応時間や正答率、表情など目に見える反応を使いますが、発汗や視線など、目に見えない反応などを使って心を読む心理学的研究を紹介します。

<参加登録>

右のQRコードを読み取り、参加登録フォームへアクセスの上、必要事項を記入してお申し込みください。



受付期間： 5/26（火）～ 7/16（木）

<https://forms.office.com/r/JRmJe3qNmh>

早期研究体験プロジェクト
（対象：学部1・2年次）

本プログラムに興味のある学生は、先端データ科学研究センターまでお問い合わせください。

随時
募集中

<お問い合わせ>

東京情報大学 総合情報学部
先端データ科学研究センター
ads@rsch.tuis.ac.jp

